

Application

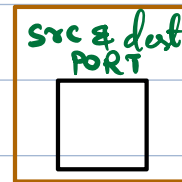
{ Application
Presentation
Session.



Application
Data



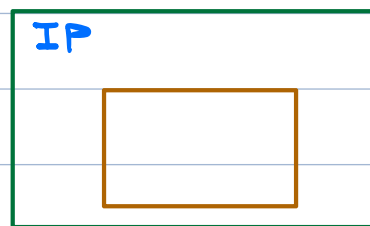
Transport



Segment



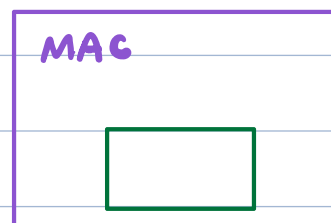
Network



Network packet



Data link
layer.

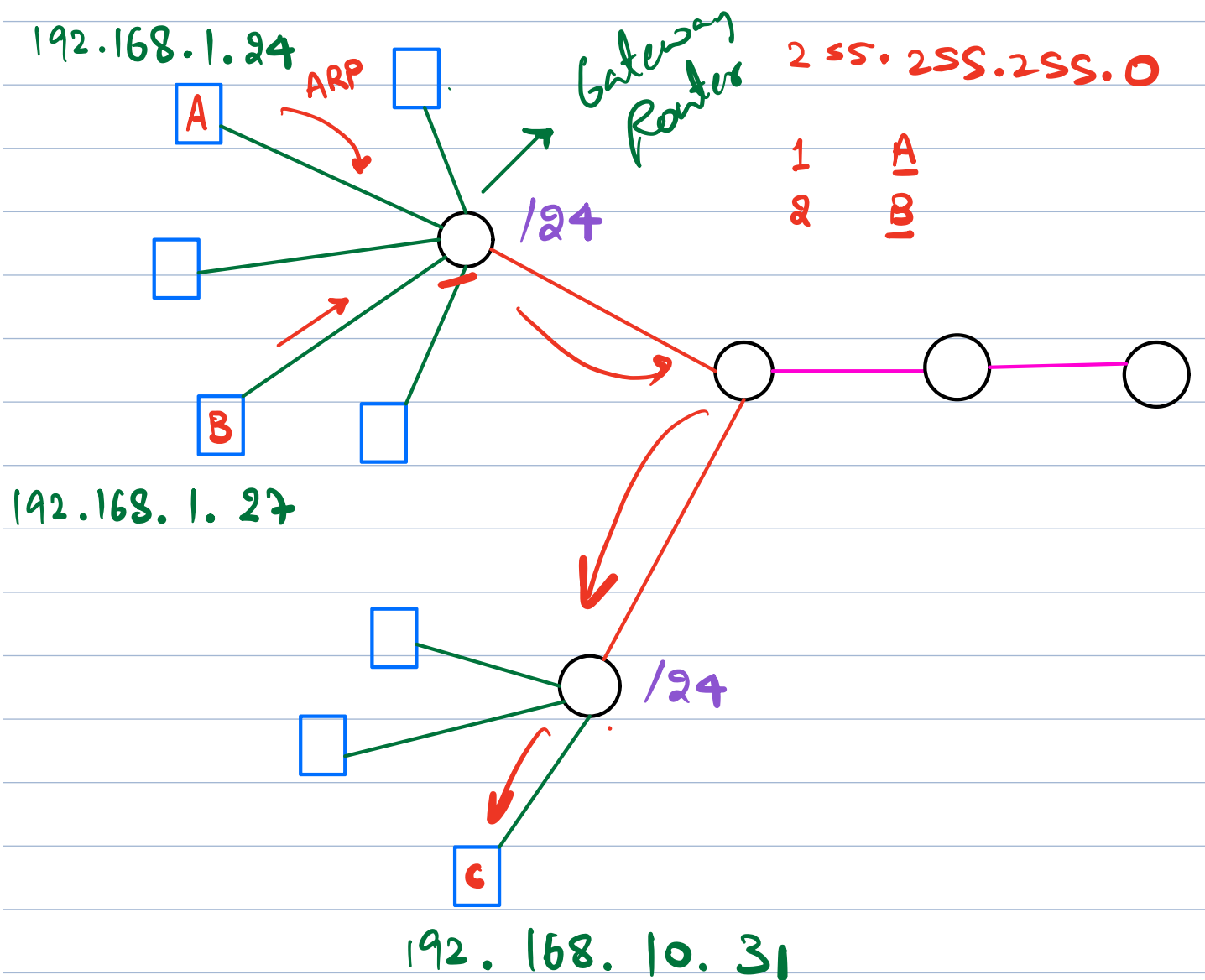


Data frame



Transmitted

Physical Layer.



IPv6

128 bits = 2^{128} or 3.4×10^{38} addresses

8 groups of 4 hex digits separated by colons

2001:0db8:85a3:0000:0000:8a2e:0370:7334

Compression Rules

1) Omit leading zeroes in each group

0d58 $\Rightarrow$ d58

0000 $\Rightarrow$ 0

2001:d58:85a3:0:0:8a2e:37d:7334

2) Replace one sequence of consecutive zero groups with ::

2001:d58:85a3::8a2e:37d:7334
fe80:: 60b1:c4ff:fe9b:878e

Common IPv6 Addresses

Address	Meaning
::1	Loopback (like 127.0.0.1)
::	Unspecified / any
fe80::/10	Link-local (auto-configured, not globally routed)
ff00::/8	Multicast
2000::/3	Global unicast (public internet)

IPv4 vs IPv6 Comparison

Feature	IPv4	IPv6
Size	32 bits	128 bits
Format	Dotted decimal	Colon-hex
Total addresses	~4.3 billion	3.4×10^{38}
Header	Variable (20–60 bytes)	Fixed 40 bytes
Broadcast	Yes	No (uses multicast)
NAT	Necessary	Not needed
ARP	Yes	Replaced by NDP
Fragmentation	Routers + hosts	Source host only

Network As a Graph

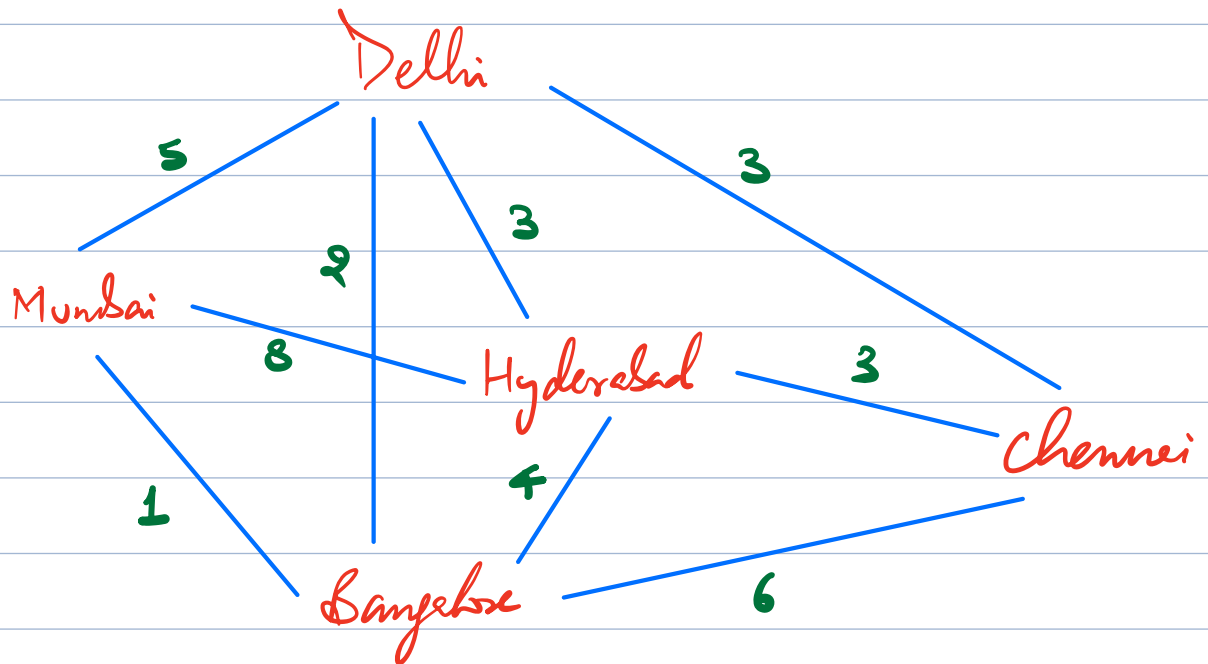
Graph $\Rightarrow$ collection of nodes & edges connecting them.

Vertices $\Rightarrow$ Routers / Network Devices

Edges $\Rightarrow$ Network links b/w nodes.

Edge weights $\Rightarrow$ latency, bandwidth, hop cost

Graph Representation



Adjacency List

Map < String , Map < String, int > > graph

D : { (M, 5), (B, 2), (H, 3), (C, 3) }

M : { (D, 5), (B, 1), (H, 8) }

B : { (M, 1), (D, 2), (H, 4), (C, 6) }

H : { (D, 3), (M, 8), (B, 4), (C, 3) }

C : { (D, 3), (H, 3), (B, 6) }

Adjacency Matrix

	D	M	B	H	C
D	0	5	2	3	3
M	5	0	1	8	0
B	2	1	0	4	6
H	3	8	4	0	3
C	3	0	6	3	0

Graph Traversal

1) BFS (Breadth first Search)

Code

```
HashSet <Node> vst;  
Queue <Node> q;  
HashMap <Node, int> ans;
```

```
vst.insert (src);  
ans.insert ( <src, 0>);  
q.enqueue (src);
```

```
while (! q.isEmpty()) {
```

```
    Node x = q.dequeue();  
    dist d = ans.get(x);
```

```
    for (all Node u connected to x) {
```

```
        if (! vst.contains(u)) {
```

```
            vst.insert(u);
```

```
            ans.insert(u, d+1);
```

```
            q.enqueue(u);
```

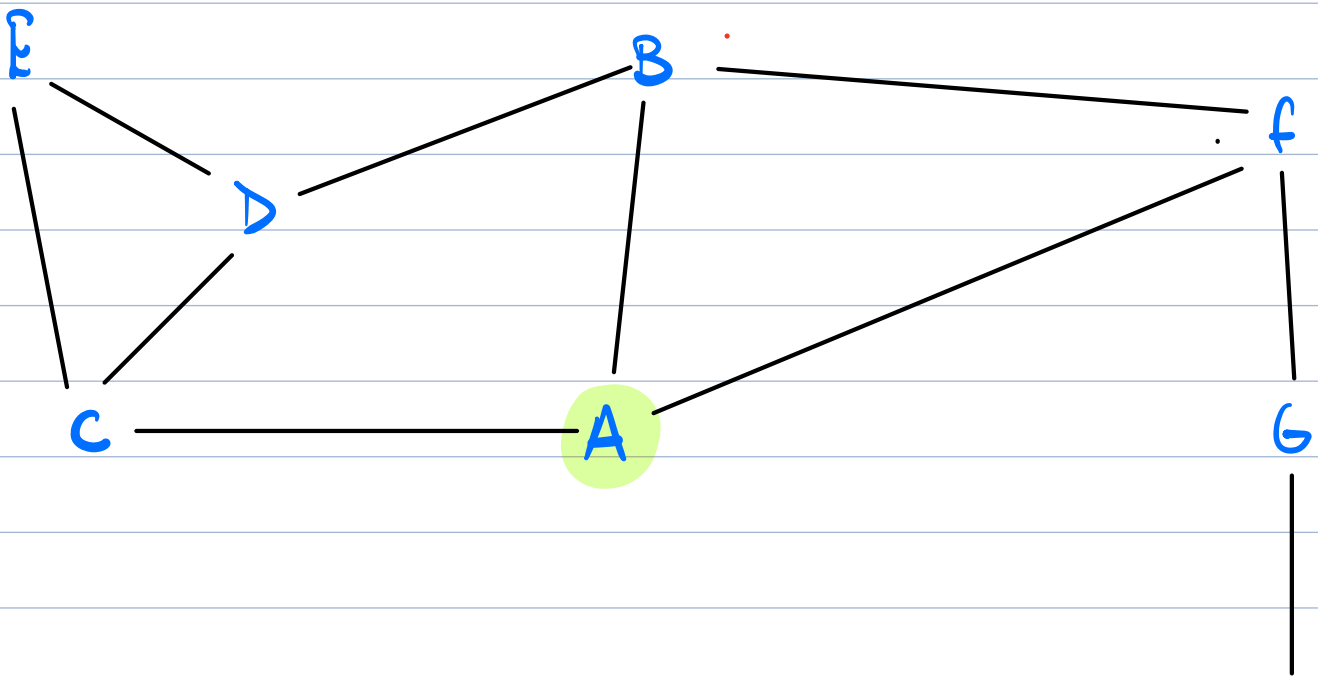
```
        }
```

```
    }
```

```
}
```

T.C. = $O(V + E)$

BFS With Path Finding



H

H.W. $\Rightarrow$ Dry Run of BFS (Shortest dist)

$\Rightarrow$ Path tracing